通用近似定理与单隐藏层前馈神经网络

前向计算、反向偏导、激活函数与优化方法

李栋, leslie_ld@126.com
博客：https://leslielee.blog.csdn.net
GitHub：https://github.com/leslielee619

2026 年 11 月 8 日

1 通用近似定理

设标量非线性函数 $\phi:\mathbb{R}\to\mathbb{R}$。当它作用于向量时采用逐元素（element-wise）操作：

$$\phi\big([z_1,\dots,z_M]^T\big)=\big[\phi(z_1),\dots,\phi(z_M)\big]^T,\qquad z_i\in\mathbb{R}. \tag{1}$$

单隐藏层前馈网络表示为

$$g(x)=v^T\phi(Wx+b)+c,\qquad x\in\mathbb{R}^D,\ W\in\mathbb{R}^{M\times D},\ b\in\mathbb{R}^M,\ v\in\mathbb{R}^M,\ c\in\mathbb{R}. \tag{2}$$

即输入层 D 维、隐藏层 M 维、输出层 1 维的网络结构。

[通用近似定理[1]，Universal Approximation Theorem] 对连续函数空间中的任意函数 $f\in\mathcal{C}([a,b]^D;\mathbb{R})$，存在 ϕ,W,b,v,c，使得

$$\forall x\in[a,b]^D,\qquad \big|f(x)-g(x)\big|<\varepsilon,$$

其中 ε 是极小正数。

对于多输出函数 $f\in\mathcal{C}([a,b]^D;\mathbb{R}^K)$ 的近似，可采用下面两种方式。

1. **每个输出用一个单输出前馈神经网络近似**

   $$g_i(x)=v_i^T\phi(W_ix+b_i)+c_i,\qquad x\in\mathbb{R}^D,\ W_i\in\mathbb{R}^{M_i\times D},\ b_i\in\mathbb{R}^{M_i},\ v_i\in\mathbb{R}^{M_i},\ c_i\in\mathbb{R}, \tag{3}$$

   共 $\sum_{i=1}^K\big(M_iD+M_i+M_i+1\big)$ 个参数。
2. **用一个共享隐藏层的多输出前馈神经网络近似**

   $$g(x)=V\phi(Wx+b)+c,\qquad x\in\mathbb{R}^D,\ W\in\mathbb{R}^{M\times D},\ b\in\mathbb{R}^M,\ V\in\mathbb{R}^{K\times M},\ c\in\mathbb{R}^K, \tag{4}$$

   共 $MD+M+KM+K$ 个参数。当 K 较大时，共享隐藏层显著减少了参数数量。

[1]邱锡鹏. 神经网络与深度学习 [M]. 2 版. 北京: 机械工业出版社, 2026: 93.

2 偏导

N 个样本的总损失函数定义为：

$$L = \frac{1}{N}\sum_{i=1}^{N}\big(g(x_i) - f(x_i)\big)^2. \tag{5}$$

这里只考虑**单输出前馈神经网络**的偏导。为方便推导，记

$$z_i = Wx_i + b, \qquad a_i = \phi(z_i), \qquad e_i = g(x_i) - f(x_i),$$

并将 $g(x_i)$ 记作 g_i，$f(x_i)$ 记作 f_i。拟合函数可表示为

$$g_i = v^T a_i + c. \tag{6}$$

各量的类型如下：W 为矩阵；x_i, b, z_i, a_i, v 均为列向量；g_i, L 为标量。

求导约定

1. 标量 a 对向量 b 求导，结果的维度应为 $\dim(b) \times 1$（即与 b 同型的列向量）。
2. 向量 a 对向量 b 求导，结果的维度应为 $\dim(b) \times \dim(a)$：每列是某个 a_i 对所有 b_j 求导组成的列向量，每行是所有 a_i 对某个 b_j 求导组成的行向量。

2.1 计算损失函数对输入的偏导

链条：$L \to g_i \to a_i \to z_i \to x_i$。逐项求导得

$$\begin{aligned}
&\frac{\partial L}{\partial g_i} = \frac{2e_i}{N}, && \\
&\frac{\partial g_i}{\partial a_i} = v, && \text{（标量对列向量求导，结果仍是列向量）} \\
&\frac{\partial a_i}{\partial z_i} = \operatorname{diag}\big(\phi'(z_i)\big), && \text{（M 维列向量对 M 维列向量求导，得 $M \times M$ 矩阵）} \\
&\frac{\partial z_i}{\partial x_i} = W^T. && \text{（M 维列向量对 D 维列向量求导，得 $D \times M$ 矩阵）}
\end{aligned} \tag{7}$$

其中 $\operatorname{diag}(\phi'(z_i))$ 的推导为

$$\frac{\partial a_{i,j}}{\partial z_{i,k}} = \frac{\partial\,\phi(z_{i,j})}{\partial z_{i,k}} = \phi'(z_{i,j})\,\frac{\partial z_{i,j}}{\partial z_{i,k}} = \phi'(z_{i,j})\,\delta_{jk},$$

δ_{jk} 为 Kronecker 符号。

由链式法则计算 $\dfrac{\partial L}{\partial x_i}$：

$$\frac{\partial L}{\partial x_i} = \frac{\partial L}{\partial g_i}\cdot\frac{\partial g_i}{\partial a_i}\cdot\frac{\partial a_i}{\partial z_i}\cdot\frac{\partial z_i}{\partial x_i} = \frac{2e_i}{N}\,W^T\operatorname{diag}\big(\phi'(Wx_i+b)\big)v. \tag{8}$$

改用 Hadamard 积（记 $\odot$ 为逐元素乘）可写成更紧凑的形式：

$$\frac{\partial L}{\partial x_i} = \frac{2e_i}{N}\,W^T\big[v \odot \phi'(Wx_i+b)\big]. \tag{9}$$

由链式法则计算分量 $\dfrac{\partial L}{\partial x_{i,j}}$：

$$\begin{aligned}\frac{\partial L}{\partial x_{i,j}} &= \sum_{m=1}^{M}\sum_{n=1}^{M}\frac{\partial L}{\partial g_i}\frac{\partial g_i}{\partial a_{i,m}}\frac{\partial a_{i,m}}{\partial z_{i,n}}\frac{\partial z_{i,n}}{\partial x_{i,j}} \\ &= \sum_{m=1}^{M}\sum_{n=1}^{M}\frac{2e_i}{N}\,v_m\,\phi'(z_{i,m})\,\delta_{mn}\,W_{nj} \qquad (10)\\ &= \frac{2e_i}{N}\sum_{m=1}^{M}v_m\,\phi'(z_{i,m})\,W_{mj}, \qquad (11)\end{aligned}$$

2.2 计算损失函数对参数的偏导

链条：

$$L \to g_i \to v, \qquad L \to g_i \to c, \qquad L \to g_i \to a_i \to z_i \to W, \qquad L \to g_i \to a_i \to z_i \to b.$$

各部分偏导为

$$\begin{aligned}\frac{\partial g_i}{\partial v} &= a_i, & \frac{\partial g_i}{\partial c} &= 1,\\ \frac{\partial z_{i,m}}{\partial W^T_{m,:}} &= x_i, & \frac{\partial z_i}{\partial b} &= \mathrm{diag}(1) = \boldsymbol{I}_M.\end{aligned}$$

由链式法则计算损失函数对参数的偏导：

$$\frac{\partial L}{\partial v} = \sum_{i=1}^{N}\frac{2e_i}{N}\,a_i, \qquad (12)$$

$$\frac{\partial L}{\partial c} = \sum_{i=1}^{N}\frac{2e_i}{N}, \qquad (13)$$

$$\frac{\partial L}{\partial W} = \sum_{i=1}^{N}\frac{2e_i}{N}\big[v\odot\phi'(z_i)\big]x_i^T, \qquad (14)$$

$$\frac{\partial L}{\partial b} = \sum_{i=1}^{N}\frac{2e_i}{N}\big[v\odot\phi'(z_i)\big]. \qquad (15)$$

这些偏导的用途可归为三类：

1. 损失对**参数**求偏导，用于更新参数（训练网络）；
2. 损失对**输入**求偏导，用于分析输入（或其分量）对训练的影响程度；
3. 输出对**输入**求偏导，用于分析输入（或其分量）对预测的影响程度。

3 常见的激活函数及其导数

激活函数	函数形式	导数	值域
sigmoid	$s(x)=\dfrac{1}{1+e^{-x}}$	$s'(x)=s(x)\big(1-s(x)\big)$	$(0,1)$
ReLU	$\phi(x)=\max(0,x)$	$\phi'(x)=\begin{cases}1, & x>0\\ 0, & x\le 0\end{cases}$	$[0,+\infty)$
softplus	$\phi(x)=\ln\big(1+e^{x}\big)$	$\phi'(x)=s(x)$	$(0,+\infty)$
Gauss	$g(x)=\exp\Big(-\dfrac{(x-\mu)^2}{2\sigma^2}\Big)$	$g'(x)=\dfrac{\mu-x}{\sigma^2}\,g(x)$	——
Leaky ReLU	$\phi(x)=\max(\alpha x,\,x), 0<\alpha\ll 1$	$\phi'(x)=\begin{cases}1, & x>0\\ \alpha, & x\le 0\end{cases}$	$\mathbb{R}$

表 1: 常见激活函数及其导数

除表中函数外，sin 与 tanh 也常被用作激活函数；在高阶导数有需求的场合（如逼近 PDE），要求激活函数充分光滑，ReLU 因二阶导数几乎处处为零并不适用。

4 训练

梯度下降的参数更新为

$$\theta_{t+1}=\theta_t-\eta_t\,\frac{\partial L}{\partial \theta_t}, \tag{16}$$

其中 θ 泛指 W,b,v,c。

按每次更新所用样本量的不同，有三种基本形式：

$$\text{SGD（随机梯度下降）：}\quad \theta_{t+1}=\theta_t-\eta_t\frac{\partial L_i}{\partial \theta_t}, \tag{17}$$

$$\text{BGD（批量梯度下降）：}\quad \theta_{t+1}=\theta_t-\frac{1}{N}\sum_{i=1}^{N}\eta_t\frac{\partial L_i}{\partial \theta_t}, \tag{18}$$

$$\text{Mini-BGD（小批量梯度下降）：}\quad \theta_{t+1}=\theta_t-\frac{1}{|\mathcal{B}_t|}\sum_{i\in\mathcal{B}_t}\eta_t\frac{\partial L_i}{\partial \theta_t}, \tag{19}$$

其中 $\mathcal{B}_t$ 是第 t 次迭代所用的部分样本索引集合，$|\mathcal{B}_t|$ 是该集合的长度。

η_t 称为学习率，可设为常数、按步衰减、先增加到 η_0 再衰减、损失不下降时衰减等。

4.1 改进的更新公式

动量法（Momentum）

$$m_{t+1} = \beta m_t + (1-\beta)\frac{\partial L}{\partial \theta_t}, \tag{20}$$

$$\theta_{t+1} = \theta_t - \eta_t\, m_{t+1}. \tag{21}$$

Nesterov 加速梯度

$$\tilde{\theta}_{t+1} = \theta_t - \eta_t m_t, \tag{22}$$

$$m_{t+1} = \beta m_t + (1-\beta)\frac{\partial L}{\partial \tilde{\theta}_{t+1}}, \tag{23}$$

$$\theta_{t+1} = \theta_t - \eta_t m_{t+1}. \tag{24}$$

AdaGrad

$$G_{t+1} = G_t + \Big(\frac{\partial L}{\partial \theta_t}\Big)^2, \tag{25}$$

$$\eta_t = \frac{\eta}{\sqrt{G_{t+1}} + \epsilon}, \tag{26}$$

$$\theta_{t+1} = \theta_t - \eta_t \frac{\partial L}{\partial \theta_t}. \tag{27}$$

RMSProp

$$s_{t+1} = \rho s_t + (1-\rho)\Big(\frac{\partial L}{\partial \theta_t}\Big)^2, \tag{28}$$

$$\eta_t = \frac{\eta}{\sqrt{s_{t+1}} + \epsilon}, \tag{29}$$

$$\theta_{t+1} = \theta_t - \eta_t \frac{\partial L}{\partial \theta_t}. \tag{30}$$

Adam

$$m_{t+1} = \beta m_t + (1-\beta)\frac{\partial L}{\partial \theta_t}, \qquad \tilde{m}_{t+1} = \frac{m_{t+1}}{1-\beta^{t+1}}, \tag{31}$$

$$s_{t+1} = \rho s_t + (1-\rho)\Big(\frac{\partial L}{\partial \theta_t}\Big)^2, \qquad \tilde{s}_{t+1} = \frac{s_{t+1}}{1-\rho^{t+1}}, \tag{32}$$

$$\theta_{t+1} = \theta_t - \eta\, \frac{\tilde{m}_{t+1}}{\sqrt{\tilde{s}_{t+1}} + \epsilon}. \tag{33}$$

AdamW AdamW 在 Adam 的基础上加入权重衰减项：

$$\theta_{t+1} = \theta_t - \eta\Big(\frac{\tilde{m}_{t+1}}{\sqrt{\tilde{s}_{t+1}} + \epsilon} + \lambda\theta_t\Big). \tag{34}$$

5 任务

用 pytorch 训练单隐藏层前馈网络，分别拟合连续函数、偏微分方程 (PDE) 以及离散傅里叶变换 (DFT)。

5.1 连续函数

取 $\mu = 1, \sigma = 1$ 的一维高斯函数作为待拟合目标：

$$f(x) = \exp\Big(-\frac{(x-1)^2}{2}\Big), x \in [0, 2]. \tag{35}$$

5.2 离散傅里叶变换

N 维离散信号 $x = [x_0, \ldots, x_{N-1}]$ 的 DFT 定义为

$$y_k = \sum_{n=0}^{N-1} x_n \exp\Big(-\frac{i2\pi kn}{N}\Big). \tag{36}$$

写成矩阵形式 $y = Wx$，其中 W 为对称矩阵：

$$W_{k,n} = \exp\Big(-\frac{i2\pi kn}{N}\Big) = \cos\Big(\frac{2\pi kn}{N}\Big) - i\sin\Big(\frac{2\pi kn}{N}\Big). \tag{37}$$

$$W = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega^1 & \omega^2 & \cdots & \omega^{N-1} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & \omega^{2(N-1)} & \cdots & \omega^{(N-1)^2} \end{bmatrix}, \omega = exp(-\frac{i2\pi}{N}) \tag{38}$$

实信号 DFT 的共轭对称性： $y_{N-k} = y_k^\star$

证明.

$$\begin{aligned} y_{N-k} &= \sum_{n=0}^{N-1} x_n \exp\Big(-\frac{i2\pi(N-k)n}{N}\Big) \\ &= \sum_{n=0}^{N-1} x_n \exp\Big(-\frac{i2\pi Nn}{N}\Big) \exp\Big(\frac{i2\pi kn}{N}\Big) \\ &= \sum_{n=0}^{N-1} x_n \exp\Big(\frac{i2\pi kn}{N}\Big) \\ &= \sum_{n=0}^{N-1} x_n [\exp\Big(-\frac{i2\pi kn}{N}\Big)]^\star \\ &= [\sum_{n=0}^{N-1} x_n \exp\Big(-\frac{i2\pi kn}{N}\Big)]^\star = y_k^\star. \end{aligned} \tag{39}$$

□

实偶信号 DFT 的对称性：$y_{N-k} = y_k$

证明. 已知实偶信号满足 $x_n = x_{N-n}$。

$$\begin{aligned} y_{N-k} &= \sum_{n=0}^{N-1} x_n \exp\Big(-\frac{i2\pi(N-k)n}{N}\Big) \\ &= \sum_{n=0}^{N-1} \frac{x_n + x_{N-n}}{2} \exp\Big(-\frac{i2\pi(N-k)n}{N}\Big) \\ &= \sum_{n=0}^{N-1} \frac{x_n + x_{N-n}}{2} \Big(\cos\Big(\frac{2\pi kn}{N}\Big) - i\sin\Big(\frac{2\pi kn}{N}\Big)\Big). \end{aligned} \tag{40}$$

$$\begin{aligned} \mathrm{Im}\big(y_{N-k}\big) &= -\sum_{n=0}^{N-1} \frac{x_n + x_{N-n}}{2} \sin\Big(\frac{2\pi kn}{N}\Big) \\ &= -\sum_{n=0}^{N-1} \frac{x_n}{2} \sin\Big(\frac{2\pi kn}{N}\Big) - \sum_{n=0}^{N-1} \frac{x_{N-n}}{2} \sin\Big(\frac{2\pi kn}{N}\Big) \\ &= -\sum_{n=0}^{N-1} \frac{x_n}{2} \sin\Big(\frac{2\pi kn}{N}\Big) + \sum_{n=0}^{N-1} \frac{x_{N-n}}{2} \sin\Big(\frac{2\pi k(N-n)}{N}\Big). \end{aligned} \tag{41}$$

令 $m = N - n$，则 $n : 0 \to N-1$ 对应 $m : N \to 1$，取模后 $m : 0 \to N-1$，故上式中两项相互抵消：

$$\mathrm{Im}\big(y_{N-k}\big) = 0 \quad \Longrightarrow \quad y_{N-k} = y_k.$$

□

训练

用 $g(x) = V\phi(Wx + b) + c$ 去近似 DFT（实部与虚部各占 N 维输出），其中 $x \in \mathbb{R}^N, g(x) \in \mathbb{R}^{2N}, V \in \mathbb{R}^{2N \times M}$。

考虑实信号的共轭对称性，参数可缩减为：$V \in \mathbb{R}^{2(\lfloor N/2 \rfloor + 1) \times M}$; 若信号还是偶对称的，可进一步缩减为：$V \in \mathbb{R}^{(\lfloor N/2 \rfloor + 1) \times M}$。

训练时限制输入 $x \in [-1, 1]$。

5.3 偏微分方程

一维热传导方程及定解条件为

$$\begin{aligned} \frac{\partial u}{\partial t} &= \alpha \frac{\partial^2 u}{\partial x^2}, && x \in [0, L],\ t \in [0, T], \\ u(x, 0) &= \sin\Big(\frac{\pi x}{L}\Big), && \text{（初始条件）} \\ u(0, t) &= u(L, t) = 0, && \text{（齐次 Dirichlet 边界条件）} \end{aligned}$$

其解析解为

$$u(x, t) = \exp\Big(-\frac{\alpha \pi^2 t}{L^2}\Big) \sin\Big(\frac{\pi x}{L}\Big). \tag{42}$$

实验中取 $L = 1,\ T = 0.3$，热扩散系数 $\alpha = 1$，则解析解简化为

$$u(x,t) = \exp\bigl(-\pi^2 t\bigr)\sin(\pi x), \qquad x \in [0,1],\ t \in [0,0.3]. \tag{43}$$

训练

用 $g(x,t) = v^T\phi(W_1 x + W_2 t + b) + c$ 去近似 PDE 的解。[2]

两种训练方式：

1. 匹配解析解——但解析解通常难以获取，甚至可能不存在；
2. 匹配微分方程本身以及边界条件、初始条件（即物理信息驱动的训练）。

第二种训练方式所需的各项及损失函数为

$$\partial_t g = v^T\bigl(W_2 \odot \phi'(z)\bigr), \tag{44}$$

$$\partial_x g = v^T\bigl(W_1 \odot \phi'(z)\bigr), \tag{45}$$

$$\partial_{xx} g = v^T\bigl((W_1 \odot W_1) \odot \phi''(z)\bigr), \tag{46}$$

$$g(x,0) = v^T\phi(W_1 x + b) + c, \tag{47}$$

$$g(0,t) = v^T\phi(W_2 t + b) + c, \tag{48}$$

$$g(1,t) = v^T\phi(W_1 + W_2 t + b) + c, \tag{49}$$

$$\begin{aligned}\mathrm{Loss} = \frac{1}{N}\sum_{i=1}^{N}\Bigl[&\bigl(\partial_t g(x_i,t_i) - \partial_{xx} g(x_i,t_i)\bigr)^2 \\ &+ \lambda_{\text{init}}\bigl(g(x_i,0) - \sin(\pi x_i)\bigr)^2 \\ &+ \lambda_{\text{low}}\, g(0,t_i)^2 + \lambda_{\text{high}}\, g(1,t_i)^2\Bigr].\end{aligned} \tag{50}$$

[2]本文记号在不同语境中略有混用，需按上下文理解；此处的 $x, t \in \mathbb{R}^M$ 表示所有采样点构成的向量。